



KRIZKODEZ

PowerShell im Unternehmen

Best Practices
Regeln
Tipps

Autor: Christoph Rust
Version: 1.0
Datum: 13.04.2026

Inhaltsverzeichnis

Inhaltsverzeichnis.....	I
Dokumentenänderungen.....	II
Verweise.....	III
Bemerkungen.....	IV
Einleitung.....	1
Skript Aufbau.....	2
Controller.....	2
INFO HEADER.....	2
COMMENT BASED HELP.....	5
PARAMETERS.....	6
PREREQUISITES.....	6
INCLUDE LIBRARIES.....	7
PARAMETER CHECK.....	8
DECLARATIONS AND DEFINITIONS.....	8
MAIN CODE.....	9
FUNCTIONS.....	10
SCRIPTBLOCKS.....	10
EXCEPTION HANDLING.....	10
Bibliotheken.....	11
Private Bibliothek.....	11
INFO HEADER.....	11
DECLARATIONS AND DEFINITIONS.....	13
SCRIPTBLOCKS.....	14
FUNCTIONS.....	14
Öffentliche Bibliothek.....	14
INFO HEADER.....	14
DECLARATIONS AND DEFINITIONS.....	16
SCRIPTBLOCKS.....	17
FUNCTIONS.....	17
Dateinamen.....	19
Controller.....	19
Private Funktionsbibliothek.....	19
Öffentliche Funktionsbibliothek.....	20
Code.....	21
Formatierung.....	28
Kommentare.....	32
Konstanten.....	33
Variablen.....	34
Funktionen.....	36
FUNCTION HELP.....	37
PARAMETERS.....	39
PARAMETER CHECK.....	39
DECLARATIONS AND DEFINITIONS.....	40
FUNCTION CODE.....	41
Parameter.....	43
Versionierung.....	46
Konfiguration.....	47

Dokumentenänderungen

Dieser Abschnitt wird in zukünftigen Versionen eine Übersicht der Korrekturen bzw. Änderungen jeder neuen Version des Standards enthalten.

Verweise

Die hier aufgeführten Verweise sind Quellen, die bei der Erstellung dieses Dokumentes genutzt wurden bzw. bei denen weiterführende Informationen zu finden sind. Jeder Verweis wird im Dokument durch eine Abkürzung der Form [NAME] repräsentiert.

[PESTEMPLATES]

Code-Vorlagen des Autors zur Anwendung des EPS (Enterprise PowerShell Standard).

<https://github.com/KrizKodez/EnterprisePowerShellStandard>

[SEMVERS]

Schema zur Versionierung von Quellcode dem z.B. die Microsoft PowerShell Gallery aber auch viele andere folgen.

<https://semver.org/>

[RFCKEYWORD]

Schlüsselwörter zum Kennzeichnen von Anforderungen verschiedener Stufen.

<https://www.rfc-editor.org/rfc/rfc2119>

[PSPRACTICESANDSTYLE]

Sammlung von Best Current Practices von Don Jones und der PowerShell Community.

<https://poshcode.gitbook.io/powershell-practice-and-style>

[WRITEBETTERPOWERSHELL]

Tipps und Tricks zur Arbeit mit Quellcode in PowerShell von 'saidbrandon'.

Anm.: Dem Autor ist nur der spiceworks Login-Name bekannt.

<https://community.spiceworks.com/t/write-better-powershell-scripts/1013904>

[SETRICTMODE]

PowerShell Kommando zum Durchsetzen strengerer Coderegeln.

<https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/set-strictmode?view=powershell-5.1>

[TYPEACCEL]

Microsoft Dokumentation der verfügbaren Type Accelerators

https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_type_accelerators

[MSAPPRVERB]

Microsoft Dokumentation der erlaubten Verben.

<https://learn.microsoft.com/en-us/powershell/scripting/developer/cmdlet/approved-verbs-for-windows-powershell-commands>

[MSAUTOVAR]

Microsoft Dokumentation der automatischen Variablen.

https://learn.microsoft.com/de-de/powershell/module/microsoft.powershell.core/about/about_automatic_variables

[PSKEYWORDS]

Microsoft Dokumentation der existierende PowerShell Schlüsselwörter.

https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_language_keywords?view=powershell-7.6

Bemerkungen

Im folgenden Dokument werden wir den Namen des Standards mit **EPS** (Enterprise PowerShell Standard) abkürzen.

Der hier vorliegenden Standard spiegelt die vom Autor gesammelten Erfahrungen und Praktiken aber auch einige eigenständige neue Ideen wieder. Trotzdem gibt es viele Übereinstimmungen zu Regelwerken von anderen Autoren, die bewusst oder auch unbewusst übernommen wurden. Deswegen erhebt der Autor natürlich keinen Anspruch auf eine irgendwie geartete Urheberschaft und stellt diesen Standard unter der GPL V3 zur allgemeinen Verfügung.

Zwei veröffentlichte Regelwerke deren Inhalte sich hier in Teilen auch wiederfinden werden, sind:

- + **Write better PowerShell Scripts** [WRITEBETTERPOWERSHELL]
- + **PowerShell Practice and Style** [PSPRACTICESANDSTYLE]

Beide Werke enthalten einen reichen Fundus an Best Current Practices und sind auch vom Autor als Quellen ausdrücklich empfohlen, obwohl der Autor in einigen Teilen bewusst davon abweicht. Diese Abweichungen sind im wesentlichen darauf zurückzuführen dass die primäre Zielgruppe dieses Standards in allererster Linie Administratoren in Unternehmen sein sollen und nicht Entwickler im allgemeinen. Zwei wesentliche Abweichungen zu den hier angegebenen Quellen sind:

1. Der Standard beinhaltet eigene **Code Vorlagen** [PESTEMPLATES] die zwingend genutzt werden müssen.
2. Der Standard verwendet sprachlich zwingende Formulierungen in Übereinstimmung mit dem **RFC2119** [RFCKEYWORD].

Die Erfahrung des Autors bei vielen Kunden hat gezeigt dass Vorgaben die nicht präzise oder zwingend genug formuliert sind zu einem ungewollt breiten Spektrum an Interpretationen und zeitraubenden Diskussionen führen, die für das Erreichen des Ziels abträglich sind. Daher werden definierte Schlüsselwörter in Großbuchstaben eingesetzt, die in Übereinstimmung mit RFC2119 interpretiert werden MÜSSEN. Für das hier vorliegende deutsche Dokument sind das die folgenden.

MUSS ERFORDERLICH NÖTIG	Die Definition ist eine absolute Anforderung des Standards.
DARF NICHT VERBOTEN	Die Definition ist ein absolutes Verbot des Standards.
SOLL EMPFOHLEN	In speziellen Umständen kann es gute Gründe geben vom Standard in diesem Punkt abzuweichen. Natürlich müssen die Auswirkungen voll und ganz verstanden und sorgfältig abgewägt werden.
SOLL NICHT NICHT EMPFOHLEN	In speziellen Umständen kann es gute Gründe geben, dass ein bestimmtes Vorgehen doch akzeptat-

	bel, ja sogar nützlich sein kann. Natürlich müssen die Auswirkungen voll und ganz verstanden und sorgfältig abgewägt werden.
DARF KANN NICHT NÖTIG OPTIONAL	Die Definition erlaubt das ein Vorgehen als optional angesehen wird.

Der EPS in der aktuellen Form berücksichtigt keine PowerShell Module da nach der Erfahrung des Autors diese in Unternehmen praktisch niemals erstellt werden. Die hier definierten Bibliotheksdateien stellen dafür eine leichtgewichtige Alternative für modularen und wiederverwendbaren Code zur Verfügung.

Der EPS in der aktuellen Form berücksichtigt ebenfalls nicht die speziellen Eigenheiten bei der Entwicklung von Code der die PowerShell Pipeline einsetzt. Weder definiert er spezielle Regeln, die für diese Situation gelten sollen noch ist der Standard dahingegen geprüft worden ob die vorgegebenen Code-Vorlagen und Regeln überhaupt in diesem Szenario funktionieren bzw. sinnvoll sind. Die Erfahrung des Autors aber hat gezeigt dass in Unternehmen sehr selten oder überhaupt nicht, PowerShell Code in Hinblick auf die spezielle Nutzung in einer Pipeline entworfen und entwickelt wird. Nichtsdestotrotz könnten trotzdem viele der hier gemachten Vorschläge und Regeln, vor allen Dingen was die Dokumentation von Metadaten betrifft, auch für diesen Typ von Code hilfreich sein.

Die hier getroffenen Vorgaben sollen nur bei Skripten zur Anwendung kommen und nicht beim interaktiven Einsatz der PowerShell.

Im Dokument werden Platzhalter in der folgenden Form verwendet:

<Name>	Verpflichtender Wert/Item vom Typ Name
[Name]	Optionaler Wert/Item vom Typ Name.
{Name}	Mehrfacher optionaler Wert/Item vom Typ Name

Einleitung

Die Microsoft PowerShell ist ein mächtiges Werkzeug für das interaktive Arbeiten sowie die Erstellung von Skripten. Beim Arbeiten in einer interaktiven Sitzung möchte der Benutzer möglichst schnell und effizient arbeiten und es ist daher jede Unterstützung von PowerShell wie kurze Parameternamen, Aliase oder komplexe und leistungsfähige Befehlsfolgen in der Pipeline willkommen. Die Anforderungen an ein Skript sind dagegen völlig andere, da ein Skript normalerweise wiederverwendet werden soll und der Code von anderen gelesen, verstanden und gepflegt werden muss. Diese entscheidenden Unterschiede lassen es sinnvoll erscheinen, Regeln und Empfehlungen zu definieren, die helfen können, die folgenden Ziele zu erreichen:

- + Strukturierte Code
- + Konsistentes Format
- + Verbesserung der Lesbarkeit
- + Vermeidung von Fehlern
- + Eine nutzbare Dokumentation
- + Inventarisierung von Skripten
- + Wiederverwendbarkeit von Code

Wir unterscheiden zwischen zwei allgemeinen Typen von Skripten und vier verschiedenen Arten von Code-Containern. Für jeden Typ gibt es eine speziell vorgesehene Vorlage die als Ausgangspunkt zum Erstellen neuer Controller/Tools, nach dem hier vorgelegten Standard, dienen sollen.

Typ	Beschreibung
Controller	Skripts die komplexe Aufgaben bewerkstelligen und dafür auch Tools einsetzen. + Controller mit Funktionen + Controller ohne Funktionen
Tool	Wiederverwendbarer Code in Funktionen, der eine dedizierte Aufgabe durchführt und in Funktionsbibliotheken bereitgestellt wird, um von Controllern genutzt zu werden. + Private Funktionsbibliothek + Öffentliche Funktionsbibliothek

Skript Aufbau

Die verschiedenen Code Dateien haben unterschiedliche Einsatzszenarien und daher unterscheiden sie sich auch in ihrem Aufbau. Dieser Aufbau soll hier im Detail beschrieben werden.

Controller

Der Controller Typ macht die „schwere Arbeit“ und ist gewissermaßen das Frontend einer Lösung deswegen sollte hier möglichst viel Wert auf Lesbarkeit, Einheitlichkeit und Wartbarkeit gelegt werden. Wir unterscheiden nach Controllern die Funktionen in der eigenen Datei definieren und solchen die entweder keine Funktionen nutzen oder diese aus einer externen Bibliothek importieren.

Struct-1

Für einen Controller MUSS eine der beiden folgenden Vorlagen genutzt werden:

- + ControllerWithFunctionsTemplate.ps1
- + ControllerWithoutFunctionsTemplate.ps1

Struct-2

Ein Controller MUSS nach dem folgenden definierten Schema erstellt werden und alle damit verbunden Regeln des Aufbaus beinhalten. Der Controller besteht somit aus den folgenden Blöcken:

- + <INFO HEADER>
- + <COMMENT BASED HELP>
- + <PARAMETERS>
- + <PREREQUISITES>
- + <INCLUDE LIBRARIES>
- + <PARAMETER CHECK>
- + <DECLARATIONS AND DEFINITIONS>
- + <MAIN CODE>
- + [FUNCTIONS]
- + [SCRIPTBLOCKS]
- + <EXCEPTION HANDLING>

INFO HEADER

Dieser Block ist ein mehrzeiliger PowerShell `<#PSScriptInfo ... #>` Kommentar und beinhaltet wichtige Metadaten wie eine eindeutige Skript-ID, die Autorenschaft, Abhängigkeiten, Release-Informationen und Beschreibungs-Tags. Durch diese Dokumentation kann der Block zum Aufbau eines Firmenweiten Skript-Katalogs genutzt werden, um eine Übersicht über vorhanden PowerShell Code zu geben und mehrfache Entwicklungen zu vermeiden. Der Block besteht aus verschiedenen Sektionen, die durch ein dotted Keyword eingeleitet werden. Es gibt einzeilige und mehrzeilige Sektionen.

Struct-2.1

Der Block stellt einen wesentlichen Beitrag zur Dokumentation des Skriptes dar und MUSS angegeben werden.

Struct-2.1.1

Bei einer einzeiligen Sektion MUSS der Wert in der gleichen Zeile wie das Keyword stehen, bei mehrzeiligen Sektionen MUSS der erste Wert auf der nächsten Zeile eingerückt stehen und jeder Wert MUSS auf einer eigenen Zeile beginnen, Werte können aber auch mehrzeilig sein:

```
.<KEYWORD> [Value]    Einzeilige Sektion
.<KEYWORD>             Mehrzeilige Sektion
    {Values}
```

Struct-2.1.2

Ein Keyword MUSS in Großbuchstaben geschrieben werden.

Struct-2.1.3

Die folgenden Keywords MÜSSEN enthalten sein, auch wenn sie keinen Wert haben und MÜSSEN die für sie spezifisch definierten Regeln erfüllen.

.TYPE Controller

MUSS bei einem Controller den Wert Controller haben.

.VERSION <MAJOR>.<MINOR>.<PATCH>

Die Version MUSS durch die folgenden Werte Major, Minor und Patch festgelegt sein. Die Versionierung erfolgt nach Regeln des Semantic Versioning. Details dazu siehe Kapitel **Versionierung** und den Verweis [SEMVERS].

.TEMPLATEVERSION 1

Aktuell MUSS der Wert 1 sein.

.PLATFORM <POWERSHELLVERSION>

Die zur Entwicklung und für die Tests genutzte PowerShell Version.

.GUID <GUID>

Jedes Skript MUSS einen eindeutigen Identifizierer haben.

Die GUID MUSS in Großbuchstaben angegeben werden.

Mit dem folgenden PowerShell Kommando kann eine neue GUID erzeugt werden:

```
[guid]::NewGuid().GUID.ToUpper()
```

.AUTHOR <AUTHORID> [(External <COMPANYNAME>)]

Der Autor des Skripts MUSS in Form eines eindeutigen Identifizierer angegeben werden, bei einem externen Mitarbeiter ist weiterhin das Schlüsselwort External und der externe Firmenname zu verwenden. AUTHORID kann jeder unternehmensweit eindeutige Bezeichner sein der die Person direkt identifiziert also z.B. E-Mail, Vorname und Nachname. Das Attribut samAccountName oder eine Personalnummer sind eher zu vermeiden da damit u.U. der Autor nicht ohne eine weitere Recherche zu ermitteln wäre.

.CONTRIBUTORS

```
{<AUTHORID> [(External <COMPANYNAME>)]}
```

Liste aller weiteren Personen neben dem Autor, die an dem Script mitgearbeitet haben. Das Format ist das gleiche wie bei AUTHOR.

.COMPANYNAME <COMPANYNAME>

MUSS den Namen des Unternehmens beinhalten.

.TAGS

{TAGVALUE}

Liste von Schlagwörtern um Controller in einem Index zu katalogisieren.

.FUNCTIONS

{FUNCTIONNAME}

Dieses Keyword SOLL nur bei einem Controller mit Funktionen genutzt werden und führt alle im Controller definierten Funktionen auf.

Die Namen MÜSSEN alphabetisch aufsteigend sortiert sein.

.EXTERNALMODULEDEPENDENCIES

{MODULENAME}

Liste aller PowerShell Module die das Skript benötigt.

Die Namen MÜSSEN alphabetisch aufsteigend sortiert sein.

.REQUIREDSCRIPTS

{SCRIPTNAME}

Liste aller PowerShell Skripts die der Controller benötigt.

Die Namen MÜSSEN alphabetisch aufsteigend sortiert sein.

.EXTERNALSCRIPTDEPENDENCIES

{SCRIPTDEPENDENCY}

Liste alle Abhängigkeiten die der Controller hat.

Beispiele für Abhängigkeiten könnten sein:

- + Genutztes User Konto
- + Privilegien
- + Rechte
- + Gruppenmitgliedschaften

.REQUIREDBINARIES

{BINARYNAME}

Liste aller Binaries die der Controller benötigt.

Für jede Binary Datei MUSS der Name und das Datei-Suffix angegeben werden.

Die Namen MÜSSEN alphabetisch aufsteigend sortiert sein.

.DESCRIPTION

<DESCRIPTION>

Eine hinreichend umfangreiche Beschreibung der Aufgabe und Umsetzung. Dies stellt die Dokumentation des Controllers dar und MUSS vorhanden sein.

.RELEASENOTES

{<YYYY-MM-DD>, <VERSION>, <AUTHORID>, <CHANGEDescription>}

Dieses Keyword MUSS genutzt werden wenn keine Versionsverwaltung mittels eines CVS wie Git genutzt wird.

Für jede Änderung MUSS eine neue Zeile erstellt werden.

Die einzelnen Teile MÜSSEN durch Komma getrennt werden.

Das Datum MUSS im ISO-Format angegeben werden.

Die Version 1.0.0 ist die erste Version, die in die Produktion genommen wird und die <CHANGEDescription> MUSS den Wert Released to production enthalten.

COMMENT BASED HELP

Dieser Block ist ein mehrzeiliger Kommentar `<# ... #>`, der das eingebaute Hilfesystem der PowerShell nutzt, um eine Dokumentation bei der Nutzung des Controllers zur Verfügung zu stellen. Der Block besteht aus verschiedenen Sektionen, die durch ein dotted Keyword eingeleitet werden. Es gibt einzeilige und mehrzeilige Sektionen.

Struct-2.2

Der Block stellt einen wesentlichen Beitrag zur Bedienbarkeit des Controllers und MUSS angegeben werden.

Struct-2.2.1

Bei einer einzeiligen Sektion MUSS der Wert in der gleichen Zeile wie das Keyword stehen, bei mehrzeiligen Sektionen MUSS der erste Wert auf der nächsten Zeile eingerückt stehen und jeder Wert MUSS auf einer eigenen Zeile beginnen, Werte können aber auch mehrzeilig sein:

```
.<KEYWORD> [Value]    Einzeilige Sektion
.<KEYWORD>             Mehrzeilige Sektion
  {Values}
```

Struct-2.2.2

Ein Keyword MUSS in Großbuchstaben geschrieben werden.

Struct-2.2.3

Eine Sektion die OPTIONAL ist und keinen Wert hat DARF komplett weggelassen werden.

Struct-2.2.4

Die folgenden Keywords, optionale oder verpflichtende, SOLLEN enthalten sein und MÜSSEN die für sie hier spezifisch definierten Regeln erfüllen.

.SYNOPSIS

`<SYNOPSIS>`

Eine sehr kurze Inhaltsangabe.

.DESCRIPTION

`<DESCRIPTION>`

Diese Sektion MUSS die identische Beschreibung enthalten wie die gleichnamige Sektion im Block INFO HEADER.

Der Grund für die Wiederholung ist dass der Block COMMENT BASED HELP über das PowerShell Hilfe-System angezeigt werden kann, der Block INFO HEADER aber nur für die automatische Katalogisierung des Codes vorgesehen ist.

.INPUTS

None | {INPUT}

[You cannot pipe input to this controller.]

Die Angabe der Input-Datentypen und eine kurze Beschreibung der Semantik der Parameter.

Hat das Skript keine Parameter MUSS hier das Wort None stehen.

Unterstützt der Controller keine Pipeline-Verarbeitung MUSS der definierte Hinweis angegeben werden.

.OUTPUTS

NONE | {OUTPUT}

Beschreibung des Rückgabetyps. Hat der Controller keine Ausgabe MUSS hier None stehen.

Jeder einzelne Input- oder Output-Eintrag MUSS mindestens über zwei Angaben verfügen, den Datentyp und einen Beschreibungstext.

Beispiel:

.INPUTS

System.String

The log messages, a log type and an action type.

[You cannot pipe input to this function.]

[.NOTES]

Enthält zusätzliche Informationen, die bei der Benutzung beachtet werden sollten.

{.EXAMPLE}

Beispiel für den Aufruf des Controllers mit Parametern und der Ausgabe die u.U. zurückgegeben wird. Jedes Beispiel MUSS eine eigenen Sektion sein.

{.LINK}

Verweise zu Ressourcen, die zu diesem Controller in Beziehung stehen.

{.PARAMETER <NAME>
<DESCRIPTION>}

Die Parameter des Controllers und eine kurze Beschreibung. Hat der Parameter einen Standardwert SOLL dieser hier durch eine Formulierung wie The default value is <Value> beschrieben werden.

PARAMETERS**Struct-2.3**

Der Block enthält die Definitionen der Parameter des Controllers. Hat der Controller keine Parameter MUSS ein leerer Block eingefügt werden:

```

# PARAMETERS
# NA

andernfalls

# PARAMETERS
<ParametersDefinition>

```

Wie die <ParametersDefinition> im Detail auszusehen hat entnehme man der Regel **Para-8** im Abschnitt **Parameter**.

PREREQUISITES

Die Prerequisites führen Anweisungen aus die Vorbedingungen für den gesamten Controller darstellen dazu gehören z.B.:

- + Das Importieren von benötigten Modulen mit dem Cmdlet Import-Module
- + Die Requires Anweisungen von PowerShell
- + Die Set-StrictMode Anweisung.

Die Angabe der Vorbedingungen setzen das Fail Fast-Prinzip um z.B. durch das explizite Importieren benötigter Module, ohne Nutzung der Autoload-Funktion der PowerShell, da so direkt beim Starten des Controllers festgestellt werden kann ob ein benötigtes Modul vorhanden ist und nicht erst bei der ersten Nutzung einer Funktion aus demselben. Dies kann u.U. helfen beschädigte Systeme oder Datenverlust zu vermeiden. Zum Fail Fast-Prinzip siehe auch den Abschnitt **Code**.

Struct-2.4

Hat der Controller keine Prerequisites MUSS ein leerer Block eingefügt sein:

```
# PREREQUISITES
# NA
```

andernfalls

```
# PREREQUISITES
<PrerequisitesDefinitions>
```

Struct-2.4.1

Benötigt der Controller eine bestimmte PowerShell Version MUSS dies durch eine Requires Anweisung in der Sektion PREREQUISITES definiert werden.

INCLUDE LIBRARIES

Hier werden Bibliotheken importiert, die dem EPS-Standard erfüllen und vom Controller im weiteren benötigt werden.

Struct-2.5

Nutzt der Controller keine Bibliotheken MUSS ein leerer Block eingefügt werden:

```
# INCLUDE LIBRARIES
# PRIVATE
# NA
```

```
# PUBLIC
# NA
```

andernfalls

```
# INCLUDE LIBRARIES
# PRIVATE
. "$PSScriptRoot\ABC.lib.ps1"

# PUBLIC
. "$PSScriptRoot\GPO.Company.lib.ps1"
```

Struct-2.5.1

Es MUSS zwischen öffentlichen und privaten Bibliotheken unterschieden werden und die Sektionen PRIVATE und PUBLIC MÜSSEN eingerückt werden.

Struct-2.5.2

Die Bibliotheken MÜSSEN über das Dot-Sourcing in den Scope des Controller geladen werden.

Struct-2.5.3

Controller mit Funktionen in der eigenen Datei d.h. z.B. solche, die die EPS-Vorlage nutzen, DÜRFEN NICHT auch noch private Bibliotheken importieren.

PARAMETER CHECK

Der Parameter Check befolgt und setzt das Fail Fast-Prinzip um, durch die frühzeitige Überprüfung der dem Controller übermittelten Argumente. Zum Fail Fast-Prinzip siehe auch den Abschnitt **Code**.

Struct-2.6

Die zur Laufzeit übergebenen Parameter MÜSSEN auf Richtigkeit geprüft werden bevor der eigentliche Code abgearbeitet wird.

Struct-2.6.1

Es SOLLEN vorrangig die von der PowerShell zur Verfügung gestellten Validate-Attribute genutzt werden. Sind die Attribute in der Parameter-Deklaration hierfür nicht ausreichend MUSS die Überprüfung mittels dieses Blocks erfolgen.

Struct-2.6.2

Hat der Controller keine Parameter oder braucht keine weitere Überprüfung MUSS ein leerer Block eingefügt werden:

```
# PARAMETER CHECK
# NA
```

andernfalls

```
# PARAMETER CHECK
# Parameter A Check Description
<ParameterCheck>

# Parameter B Check Description
<ParameterCheck> ...
```

Es MUSS jeder <ParameterCheck> eingerückt und mit einem beschreibenden Kommentar eingeleitet werden.

Beispiel:

Hat ein Controller einen Parameter zum Laden einer Datei z.B. eine Konfigurationsdatei, sollte das Laden dieser Datei im PARAMETER CHECK Block erfolgen:

```
# PARAMETER CHECK
# Test if file is existing and could be loaded.
$content = Get-Content -Path $Configfile -ErrorAction Stop
```

DECLARATIONS AND DEFINITIONS

Die Definition und Deklaration von Variablen und Konstanten an einer dedizierten und prominenten Position im Controller verdeutlicht die Nutzung und Wichtigkeit der Elemente.

Struct-2.7

Hat der Controller keine globalen Variablen oder Konstanten MUSS ein leerer Block der folgenden Form eingefügt werden:

```
# DECLARATIONS AND DEFINITIONS
# ARGUMENTS
# NA

# CONSTANTS
```



```
# NA

# VARIABLES
# NA
```

andernfalls MÜSSEN Variablen, Konstanten und Argumente in getrennten Sektionen deklariert/definiert werden.

```
# DECLARATIONS AND DEFINITIONS
# ARGUMENTS
<ArgumentDefinitions>

# CONSTANTS
<ConstantDefinitions>

# VARIABLES
<VariableDefinitions>
```

Die Sektionen ARGUMENTS, CONSTANTS und VARIABLES MÜSSEN eingerückt werden. Zur Definition von Konstanten, Variablen oder Argumenten informieren die entsprechende Abschnitte **Konstanten**, **Variablen** bzw. Regel **Para-7.1** im Abschnitt **Parameter**.

MAIN CODE

Struct-2.8

Dieser Block enthält den eigentlichen Arbeitscode des Controllers und MUSS mit dem folgenden Kommentar eingeleitet werden:

```
# CONTROLLER MAIN CODE
```

Im weiteren Aufbau unterscheiden wir zwischen Controllern, die Funktionen in der eigenen Datei definieren und solchen die keine Funktionen nutzen oder sie aus einer externen Bibliothek importieren.

Controller mit Funktionen

Struct-2.8.1

Bei Controllern mit Funktionen MUSS die folgende Struktur eingehalten werden:

```
# CONTROLLER MAIN CODE
function Main
{
    <MainCode>
}# End of function Main.

# FUNCTIONS
<FunctionList>

# EXCEPTION HANDLING
# NA

. Main
```

Struct-2.8.1.1

Der Code des Controllers MUSS in einer Funktion mit dem Namen `Main` enthalten sein.

Struct-2.8.1.2

Die Funktion `Main` MUSS die erste definierte Funktion sein, direkt nach dem Kommentar `# CONTROLLER MAIN CODE` starten und mit dem Kommentar `# End of function Main` beendet werden.

Struct-2.8.1.3

Alle weiteren, von der Funktion `Main` aufgerufenen Funktionen MÜSSEN im `FUNCTIONS` Block definiert werden, der sich direkt an das Ende der `Main` Funktion anschließt.

Struct-2.8.1.4

Zusätzlich MÜSSEN die Namen aller Funktionen im Block `INFO HEADER` in der Sektion `FUNCTIONS` aufgeführt werden.

Struct-2.8.1.5

Die letzte Zeile des Controller MUSS den Aufruf der `Main` Funktion enthalten. Durch das dot-sourcing wird der Code der `Main` Funktion in den Scope des Skripts gehoben.

Controller ohne Funktionen**Struct-2.8.2**

Bei Controllern ohne Funktionen MUSS die folgende Struktur eingehalten werden:

```
# CONTROLLER MAIN CODE
<MainCode>
# END MAIN CODE

# EXCEPTION HANDLING
# NA
```

Diese Controller enthalten den Code zwischen dem Start- und dem Endkommentar.

FUNCTIONS**Struct-2.9**

Dieser Block DARF NICHT vorhanden sein wenn der Controller keine Funktionen definiert bzw. aus einer Bibliotheksdatei lädt.

SCRIPTBLOCKS**Struc-2.10**

Dieser Block ist OPTIONAL wenn der Controller Scriptblocks nutzt, andernfalls KANN ein leerer Block auch weggelassen werden.

EXCEPTION HANDLING**Struct-2.11**

Hat der Controller keinen globalen Exception-Handler MUSS ein leerer Block eingefügt werden:

```
# EXCEPTION HANDLING
# NA
```

andernfalls muss ein `trap` Block folgen

```
# EXCEPTION HANDLING
trap
{
    ...
}
```

Struct-2.12

Die Blöcke PREREQUISITES, INCLUDE LIBRARIES, PARAMETER CHECK und DECLARATIONS AND DEFINITIONS MÜSSEN NICHT in einer definierten Reihenfolge vorkommen, sie MÜSSEN aber zwischen dem Block PARAMETERS und SCRIPT MAIN CODE stehen.

Bibliotheken

Um den Code des Controllers so kurz und übersichtlich wie möglich zu halten SOLLTE der Code von benötigten Funktionen in eine oder mehrere Bibliotheksdateien ausgelagert werden. Dies erhöht die Lesbarkeit des Controllers und ermöglicht die Wiederverwendung von Code in anderen Controllern und implementiert damit das **DRY-Prinzip (Don't repeat yourself)**.

Struct-3

Für eine Bibliothek MUSS eine der beiden folgenden Vorlagen genutzt werden:

```
+ PrivateLibraryTemplate.lib.ps1
+ PublicLibraryTemplate.lib.ps1
```

Struct-4

In Bibliotheken DÜRFEN andere öffentliche oder private Bibliotheken NICHT direkt oder indirekt durch Funktionen oder Scriptblocks importiert werden.

Private Bibliothek

Eine private Bibliothek enthält ausschließlich Code, der von genau einem Controller genutzt wird. Die private Bibliothek wird also stets zusammen mit dem nutzenden Controller ausgeliefert.

Struct-5

Eine private Bibliothek MUSS nach dem im folgenden definierten Schema erstellt werden und alle damit verbunden Regeln des Aufbaus befolgen. Die private Bibliothek besteht somit aus den folgenden Blöcken:

```
+ <INFO HEADER>
+ <DECLARATIONS AND DEFINITIONS>
+ [SCRIPT BLOCKS]
+ <FUNCTIONS>
```

INFO HEADER

Zur allgemeinen Beschreibung siehe den äquivalenten Block für den Controller. Der Block besteht aus verschiedenen Sektionen, die durch ein dotted Keyword eingeleitet werden. Es gibt einzeilige und mehrzeilige Sektionen.

Struct-5.1

Der Block stellt einen wesentlichen Beitrag zur Dokumentation der Bibliothek dar und MUSS angegeben werden.

Struct-5.1.1

Bei einer einzeiligen Sektion MUSS der Wert in der gleichen Zeile wie das Keyword stehen, bei mehrzeiligen Sektionen MUSS der erste Wert auf der nächsten Zeile eingerückt stehen und jeder Wert MUSS auf einer eigenen Zeile beginnen, Werte können aber auch mehrzeilig sein:

```
.<KEYWORD> [Value]    Einzeilige Sektion
.<KEYWORD>             Mehrzeilige Sektion
    {Values}
```

Struct-5.1.2

Ein Keyword MUSS in Großbuchstaben geschrieben werden.

Struct-5.1.3

Die folgenden Keywords MÜSSEN enthalten sein, auch wenn sie keinen Wert haben und MÜSSEN die für sie spezifisch definierten Regeln erfüllen.

.TYPE Private Library

MUSS bei einer privaten Bibliothek den Wert Private Library haben.

.TEMPLATEVERSION 1

Aktuell MUSS der Wert 1 sein.

.PLATFORM <POWERSHELLVERSION>

MUSS identisch sein mit der Wert aus der entsprechenden Sektion des Controller.

.GUID <GUID>

Jede Bibliothek MUSS einen eindeutigen Identifizierer haben.

Die GUID MUSS in Großbuchstaben angegeben werden.

Mit dem folgenden PowerShell Kommando kann eine neue GUID erzeugt werden:

```
[guid]::NewGuid().GUID.ToUpper()
```

.AUTHOR

MUSS identisch sein mit der Wert aus der entsprechenden Sektion des Controller.

.CONTRIBUTORS

MUSS identisch sein mit der Wert aus der entsprechenden Sektion des Controller.

.COMPANYNAME

MUSS identisch sein mit der Wert aus der entsprechenden Sektion des Controller.

.TAGS

```
{TAGVALUE}
```

Stichwörter um Funktionen und Bibliotheken in einem Index zu finden.

.FUNCTIONS

```
{FUNCTIONNAME}
```

Diese Sektion MUSS alle in der Bibliothek definierten Funktionsnamen enthalten.

Die Namen MÜSSEN alphabetisch aufsteigend sortiert sein.

.EXTERNALMODULEDEPENDENCIES

```
{<FUNCTIONNAME>,<MODULENAME>}
```

Für jede Funktion in der Bibliothek MUSS der Namen der Funktion und das genutzte externe PowerShell Modul aufgeführt werden.
Die Einträge MÜSSEN nach den Funktionsnamen alphabetisch aufsteigend sortiert sein.

.REQUIREDSCRIPTS <CONTROLLERNAME>

Hier MUSS der Name des Controllers stehen, der diese private Bibliothek importiert.

.REQUIREDBINARIES

{<FUNCTIONNAME>,<BINARYNAME>}

Für jede Funktion in der Bibliothek MUSS der Name der Funktion und das genutzte externe Binary inklusive Datei-Suffix aufgeführt werden.

Die Einträge MÜSSEN nach den Funktionsnamen alphabetisch aufsteigend sortiert sein.

.DESCRIPTION

Diese Sektion MUSS den folgenden Text beinhalten:

```
This library contains private functions for
the script defined in REQUIREDSCRIPTS.
```

.RELEASENOTES

{<YYYY-MM-DD>,[FUNCTIONNAME], <AUTHOR>, <CHANGEDescription>}

Es gelten die selben Regeln wie im INFO HEADER Block des Controllers. Werden Änderungen an Funktionen oder Scriptblocks vorgenommen, MUSS der Name der Funktion bzw. des Scriptblocks aufgeführt werden.

DECLARATIONS AND DEFINITIONS

Die Definition und Deklaration von Variablen und Konstanten an einer dedizierten und prominenten Position in der Bibliothek verdeutlicht die Nutzung, Sichtbarkeit und Wichtigkeit der Elemente. Für weitere Regeln zur Verwendung von Konstanten und Variablen siehe die entsprechenden Abschnitte **Konstanten**, **Variablen**.

Struct-5.2

Die Deklaration und Definition von globalen Variablen und Konstanten in der privaten Bibliothek entlastet den Controller Code und SOLLTE im allgemeinen in der Bibliothek durchgeführt werden.

Struct-5.3

Hat die private Bibliothek keine globalen Variablen oder Konstanten MUSS ein leerer Block der folgenden Form eingefügt werden:

```
# DECLARATIONS AND DEFINITIONS
# CONSTANTS
# NA

# VARIABLES
# NA
```

andernfalls MÜSSEN Variablen und Konstanten in getrennten Sektionen deklariert/definiert werden.

```
# DECLARATIONS AND DEFINITIONS
```

```
# CONSTANTS
<ConstantDefinitions>

# VARIABLES
<VariableDefinitions>
```

Die Sektionen CONSTANTS und VARIABLES MÜSSEN eingerückt werden.

Zur Definition von Konstanten und Variablen informieren die entsprechende Abschnitte **Konstanten, Variablen**.

SCRIPTBLOCKS

Struct-5.4

Dieser Block ist OPTIONAL wenn die private Bibliothek Scriptblocks definiert, andernfalls KANN ein leerer Block auch weggelassen werden.

FUNCTIONS

Struct-5.5

Hat die private Bibliothek keine Funktionen MUSS ein leerer Block eingefügt werden:

```
# FUNCTIONS
# NA
```

andernfalls

```
# FUNCTIONS
<FunctionList>
```

In der <FunctionList> MUSS die Reihenfolge der Funktionsdefinitionen alphabetisch aufsteigend sortiert sein.

Öffentliche Bibliothek

Um die Wiederverwendung von Funktionen im Unternehmen zu ermöglichen und sich die doch recht aufwendige Erstellung von PowerShell Modulen zu ersparen wird die Nutzung von öffentlichen Bibliotheksdateien empfohlen.

Struct-6

Eine öffentliche Bibliothek MUSS nach dem im folgenden definierten Schema erstellt werden und alle damit verbunden Regeln des Aufbaus befolgen. Die öffentliche Bibliothek besteht somit aus den folgenden Blöcken:

```
+ <INFO HEADER>
+ <DECLARATIONS AND DEFINITIONS>
+ [SCRIPT BLOCKS]
+ <FUNCTIONS>
```

INFO HEADER

Zur allgemeinen Beschreibung siehe den äquivalenten Block für den Controller bzw. die öffentliche Bibliothek. Der Block besteht aus verschiedenen Sektionen, die durch ein dotted Keyword eingeleitet werden. Es gibt einzeilige und mehrzeilige Sektionen.

Struct-6.1

Der Block stellt einen wesentlichen Beitrag zur Dokumentation der Bibliothek dar und MUSS angegeben werden.

Struct-6.1.1

Bei einer einzeiligen Sektion MUSS der Wert in der gleichen Zeile wie das Keyword stehen, bei mehrzeiligen Sektionen MUSS der erste Wert auf der nächsten Zeile eingerückt stehen und jeder Wert MUSS auf einer eigenen Zeile beginnen, Werte können aber auch mehrzeilig sein:

```
.<KEYWORD> [Value]    Einzeilige Sektion
.<KEYWORD>             Mehrzeilige Sektion
    {Values}
```

Struct-6.1.2

Ein Keyword MUSS in Großbuchstaben geschrieben werden.

Struct-6.1.3

Die folgenden Keywords MÜSSEN enthalten sein, auch wenn sie keinen Wert haben und MÜSSEN die für sie spezifisch definierten Regeln erfüllen.

.TYPE Public Library

MUSS bei einer öffentlichen Bibliothek den Wert Public Library haben.

.TEMPLATEVERSION 1

Aktuell MUSS der Wert 1 sein.

.GUID <GUID>

Jede Bibliothek MUSS einen eindeutigen Identifizierer haben.

Die GUID MUSS in Großbuchstaben angegeben werden.

Mit dem folgenden PowerShell Kommando kann eine neue GUID erzeugt werden:

```
[guid]::NewGuid().GUID.ToUpper()
```

.FUNCTIONS

```
{<FUNCTIONNAME>, <MAJOR>.<MINOR>.<PATCH>}
```

Jede veröffentlichte Funktion der Bibliothek MUSS hier mit ihrer aktuellen Versionsnummer aufgeführt werden da innerhalb der Funktionen selber keine Versionsnummer mitgeführt werden.

Die Einträge MÜSSEN nach den Funktionsnamen alphabetisch aufsteigend sortiert sein.

Die Versionierung erfolgt nach Regeln des Semantic Versioning. Details dazu siehe Kapitel **Versionierung** und den Verweis [SEMVERS].

.AUTHORS

```
{<FUNCTIONNAME>, <AUTHORID> [(EXTERNAL <COMPANYNAME>)]}
```

Jeder veröffentlichten Funktion MUSS ein Autor oder Contributor zugewiesen werden.

Die Einträge MÜSSEN nach den Funktionsnamen alphabetisch aufsteigend sortiert sein.

.CONTRIBUTORS

```
<FUNCTIONNAME>, <AUTHORID> [(EXTERNAL <COMPANYNAME>)]}
```

Jeder veröffentlichten Funktion MUSS ein Autor oder Contributor zugewiesen werden.

Die Einträge MÜSSEN nach dem Funktionsnamen alphabetisch aufsteigend sortiert sein.

.COMPANYNAME <COMPANYNAME>

MUSS den Namen des Unternehmens beinhalten.

.TAGS

{TAGVALUE}

Stichwörter um Funktionen und Bibliotheken in einem Index zu finden.

.EXTERNALMODULEDEPENDENCIES

{<FUNCTIONNAME>,<MODULENAME>}

Für jede Funktion in der Bibliothek MUSS der Namen der Funktion und das genutzte externe PowerShell Modul aufgeführt werden.

Die Einträge MÜSSEN nach dem Funktionsnamen alphabetisch aufsteigend sortiert sein.

.EXTERNALSCRIPTDEPENDENCIES

{<FUNCTIONNAME>,<DEPENDENCY>}

Liste alle Abhängigkeiten der hier definierten Funktionen.

Beispiele für Abhängigkeiten können sein:

- + Genutztes User Konto
- + Privilegien
- + Rechte
- + Gruppenmitgliedschaften

.REQUIREDBINARIES

{<FUNCTIONNAME>,<BINARYNAME>}

Liste aller Binaries die die jeweilige Funktion benötigt.

Für jede Binary Datei MUSS der Name und das Datei-Suffix angegeben werden.

Die Einträge MÜSSEN nach dem Funktionsnamen alphabetisch aufsteigend sortiert sein.

.DESCRIPTION

{DESCRIPTION}

Kurze Beschreibung welcher Typ von Funktionen in der Bibliothek bereitgestellt wird.

.RELEASENOTES

{<YYYY-MM-DD>,<FUNCTIONNAME>,<VERSION>,<AUTHORID>,<CHANGEDescription>}

Dieses Keyword MUSS genutzt werden wenn keine Versionsverwaltung mittels eines CVS wie Git genutzt wird.

Die Einträge MÜSSEN nach den Datum aufsteigend sortiert sein.

Für jede Änderung MUSS eine neue Zeile erstellt werden.

Die einzelnen Teile MÜSSEN durch Komma getrennt werden.

Das Datum MUSS im ISO-Format angegeben werden.

Die Version 1.0.0 ist die erste Version, die in die Produktion genommen wird und die <ChangeDescription> MUSS den Wert Released to production enthalten.

DECLARATIONS AND DEFINITIONS

Die Definition und Deklaration von Konstanten an einer dedizierten und prominenten Position in der Bibliothek verdeutlicht die Nutzung, Sichtbarkeit und Wichtigkeit der Elemente. Für weitere Regeln zur Verwendung von Konstanten siehe den Abschnitt Konstanten.

Struct-6.2

Hat die öffentliche Bibliothek keine Konstanten MUSS ein leerer Block der folgenden Form eingefügt werden:

```
# DECLARATIONS AND DEFINITIONS
# CONSTANTS
# NA
```

andernfalls

```
# DECLARATIONS AND DEFINITIONS
# CONSTANTS
<ConstantDefinitions>
```

Der Codeblock enthält hier keine Möglichkeit Variablen zu deklarieren oder zu definieren da in öffentlichen Bibliotheken keine globalen Variablen erlaubt sind, siehe dazu auch Regel **Struct-12**.

SCRIPTBLOCKS**Struct-6.3**

Der Block ist OPTIONAL wenn die öffentliche Bibliothek Scriptblocks definiert andernfalls KANN ein leerer block auch weggelassen werden.

FUNCTIONS**Struct-6.4**

Hat die öffentliche Bibliothek keine Funktionen MUSS ein leerer Block eingefügt werden:

```
# FUNCTIONS
# NA
```

andernfalls

```
# FUNCTIONS
<FunctionList>
```

In der <FunctionList> MUSS die Reihenfolge der Namen alphabetisch sortiert sein.

Struct-7

Wurde eine öffentliche Bibliothek zur Nutzung freigegeben DARF der Name von Konstanten, Funktionen oder Scriptblocks NICHT mehr geändert werden.

Struct-8

Wurde eine öffentliche Bibliothek zur Nutzung freigegeben DARF eine Konstante NICHT einen neuen Wert erhalten.

Struct-9

Wurde eine öffentliche Bibliothek zur Nutzung freigegeben DARF der Code eines Scriptblocks NICHT mehr geändert werden.

Struct-10

Wurde eine öffentliche Bibliothek zur Nutzung freigegeben DARF der Code einer Funktion NICHT geändert werden wenn daraus eine neue Major-Version folgen würde. Es sind also nur Minor- oder Patch-Änderungen erlaubt. Wann eine Code-Änderung zu einer neuen Major-Version führt ist in Regel **Ver-7** beschrieben.

Struct-11

Hat eine Funktion einer öffentlichen Bibliothek eine neue Major-Version MUSS diese mit neuem Namen neben der existierenden Funktion veröffentlicht werden. Wie der Name für diese neue Funktion zu bilden ist entnehme man der Regel **Func-2.2**.

Wird z.B. für eine Funktion New-CompanyUser eine neue Major-Version veröffentlicht muss diese den neuen Namen New-CompanyUserV2 tragen. Die Nutzer der öffentlichen Bibliothek haben so die Möglichkeit ihren Code u.U. auf die neue Version umzustellen und laufen nicht Gefahr das existierender Code zu Problemen führt.

Struct-12

In öffentlichen Bibliotheken DÜRFEN globale Variablen NICHT deklariert/definiert werden um die Gefahr von Seiteneffekten zu vermeiden.

Dateinamen

Die Regeln zur Benennung einer Codedatei unterscheiden sich jeweils zwischen den hier vorgestellten drei Typen, da an den Namen, abhängig vom Typ des Skripts, unterschiedliche Anforderungen gestellt werden.

Controller

Controller sind die „Arbeitspferde“ und führen im Gegensatz zu Funktionen komplizierte und oft mehrere durchaus verschiedene Tätigkeiten aus.

Name-1

Ein Controller KANN beliebig benannt werden, der Name SOLL so beschreibend wie möglich und dabei ein Kompromiss aus Verständlichkeit und Länge sein.

Name-2

Bei Namen des Controllers SOLL NICHT die Microsoft Verb-Noun Konvention genutzt werden, da dieser Typ von Skript i.a. komplexe Aufgaben bearbeitet, die nicht durch dieses einfache Namensschema zufriedenstellend abgebildet werden können.

Private Funktionsbibliothek

Private Funktionsbibliotheken sind einem dediziertem Controller beigelegt, daher steht ihr Name in Beziehung zu diesem Controller.

Name-3

Der Name einer privaten Bibliothek MUSS nach dem folgenden Schema aufgebaut sein:

`<ControllerName>.lib.ps1`

Durch das Einschließen des Controller Namens wird die exklusive Bindung der Bibliothek nochmals verdeutlicht.

Name-4

Sollte ein Controller zwei oder mehr private Bibliotheken nutzen MUSS der Stammmname aus der Regel **Name-3** hierarchisch um einen eindeutigen Bezeichner für jede Bibliothek erweitert werden:

`<Lib1>.<ControllerName>.lib.ps1`

Dabei können z.B. die privaten Funktionen thematisch in verschiedenen Bibliotheken organisiert und die Bezeichner entsprechend gewählt werden.

Beispiel:

	Logging.Controller.lib.ps1 Controller.lib.ps1
	Logging.lib.ps1

Öffentliche Funktionsbibliothek

Öffentliche Funktionsbibliotheken werden im Unternehmen gemeinsam genutzt, daher steht der Name in Beziehung zu diesem Unternehmen.

Name-5

Der Name einer öffentlichen Bibliothek MUSS eindeutig in der Menge aller veröffentlichten Bibliotheken sein.

Name-6

Der Name einer öffentlichen Funktionsbibliothek MUSS den Namen des Unternehmens beinhalten und nach dem folgenden Schema aufgebaut sein:

`<Kategorie>.<CompanyName>.lib.ps1`

Die Funktionen SOLLTEN nach klar definierten Kategorien organisiert sein und die Bibliothek den Bezeichner dieser Kategorie im Namen enthalten. Ein Bezeichner für eine Kategorie DARF NICHT eine Verb-Noun Kombination enthalten.

Name-7

Sind die Kategorien hierarchisch geordnet, MUSS dies im Namen der Funktionsbibliothek durch einen voll qualifizierten Namen kenntlich gemacht werden:

`[ThemaN]. ... .[Thema2].<Thema1>.<CompanyName>.lib.ps1`

Die Hierarchie wird dabei von spezifisch nach allgemein durchlaufen und entsprechend im Namen der Bibliothek von Links nach Rechts umgesetzt. Der Teilbezeichner ThemaN MUSS also eine Unterkategorie von Thema(N-1) sein und Thema(N-1) eine Unterkategorie von Thema(N-2) etc. pp.

Beispiel:

	<code>GP0Support.ActiveDirectory.ACME.lib.ps1</code>
---	--

Code

Bei der Erstellung von Quellcode können verschiedene Prinzipien Anwendung finden, die die Lesbarkeit, Verständlichkeit und Wartbarkeit des Codes deutlich verbessern. Einige werden im Folgenden hier beschrieben.

Don't be too smart.

Sollten für bestimmte Probleme allgemeine Lösungen, Verfahren oder Standards existieren SOLLTEN diese den eigenen kreativen Neuentwicklungen vorgezogen werden.

Beispiel:

Verwendet Microsoft in vielen PowerShell Funktionen den Parameter -ComputerName für den ausgewählten Computer macht es wenig Sinn bei einer eigenen Funktion einen Parameter mit dem identischen Zweck -Host oder vielleicht -Computer zu verwenden. Dies erschwert die Nutzung der Funktion für den User unnötig da es für ihn unerwartet kommt z.B. in seiner gewohnten Arbeitsweise mit sonst üblichen Funktionen oder Schnittstellen. Dies nennt man **POLS (Principle of Least Surprise)**.

Don't Repeat Yourself (DRY).

Diese Prinzip bedeutet z.B. das Codeduplikate nicht vorkommen sollten und wird durch den Einsatz von Funktionen innerhalb eines Controllers unterbunden. Diese Prinzip kann aber auch weiter gedacht werden und bedeutet das unterschiedliche Administratoren nicht unnötig doppelten Code schreiben weil eine Funktionsbibliothek im Team oder Unternehmen angelegt und gepflegt wird.

Do not nest too much.

Tiefe Verschachtlung von Codeblöcken ist in fast allen Fällen schwer lesbar und in manchen sogar nur schwer verstehbar, daher sollte man versuchen das Verschachteln auf maximal drei Ebenen zu beschränken.

Beispiel:

	<pre>If (\$IsFirstCondition) { if (\$IsSecondCondition) { foreach (\$User in \$Users) { ... } } }</pre>
	<pre>If (\$IsFirstCondition) { if (-not \$IsSecondCondition) { continue } foreach (\$User in \$Users) { ... } }</pre>

Fail Fast

Das Fail Fast-Prinzip im Kontext dieses Standards bedeutet das innerhalb eines Codepfades zu aller erst die Bedingungen geprüft werden sollen, die für die Fortführung erfüllt sein müssen. Ein Codepfad kann dabei ein Skript, eine Funktion oder eine Kontrollstruktur sein. Dieses Prinzip wird durch verschiedenen Konstruktionen implementiert von denen drei Beispiele hier exemplarisch aufgeführt werden sollen.

+ Frühes Laden von PowerShell Modulen.

PowerShell verfügt zwar über einen Mechanismus um Module automatisch nachzuladen wenn Funktionen aus diesen ausgeführt werden sollen, allerdings verstößt dieses Verhalten gegen das Fail Fast-Prinzip, weil der Code so schnell wie möglich darüber informieren sollte ob ein benötigtes Modul überhaupt vorhanden ist. Durch das manuelle Laden der Module im Block PREREQUISITES wird dies erreicht.

Beispiel:

	# PREREQUISITES Import-Module ActiveDirectory -ErrorAction Stop
---	--

+ Frühe Überprüfung von Parametern.

Eine Möglichkeit dafür besteht z.B. in der Nutzung der PowerShell Validate-Attribute bei der Parameterdefinition:

```
[CmdletBinding()]
param
(
    [Parameter(Mandatory)]
    [ValidateSet("INFO","ERROR","WHATIF")]
    [string]$Type
)
```

+ Frühes Verlassen von Schleifen.

Zu Beginn des Schleifenkörpers werden die Bedingungen geprüft die notwendig für die Weiterbearbeitung sind:

```
foreach ($User in $Users)
{
    if ($User.Name -notmatch $REGEX_VALID_NAME) {continue}
    DoSomething -Identity $User.Name
    . . .
}
```

Code-1

Quellcode MUSS in Englisch formuliert sein.

Code-2

Quellcode SOLLTE mit einem CVS z.B. Git verwaltet werden.

Code-3

Wenn ein Problem mit PowerShell Code oder .NET Code gelöst werden kann SOLLTE die PowerShell Lösung bevorzugt werden außer es stehen dem wesentliche Gründe entgegen z.B. zu schlechte Performance.

Code-4

Es dürfen keine Batch-Skripts (*.cmd , *.bat) oder VBS-Skripts gestartet werden.

Code-5

Müssen strengere Regeln zur Vermeidung von Fehlern durchgesetzt werden SOLLTE der PowerShell Befehl Set-StrictMode verwendet werden, der die Befolgung einiger Best Current Practices zur Kodierung sicherstellt wie die folgenden:

- + Verbieta Verweise auf nicht initialisierte Variablen, einschließlich uninitialisierter Variablen in Strings.
- + Verbieta Verweise auf nicht existierende Eigenschaften eines Objekts.
- + Verbieta Funktionsaufrufe, die die Syntax für den Aufruf von Methoden verwenden.
- + Verbieta eine Variable ohne Namen (\${}).

Der Umfang der Regeln kann von der PowerShell Version abhängig sein. Für eine Liste der aktuellen Regeln siehe man [SETSTRICTMODE].

Code-6

Jede Anweisung MUSS in einer separaten Zeile beginnen, Ausnahmen sind die Schlüsselwörter return, break, continue. Diese dürfen, wenn die Zeile kurz und übersichtlich ist, kombiniert werden.

Beispiel:

	<code>Write-Host 'Function has been aborted'; return</code>
--	---

Code-7

Soll der Rückgabewert eines Cmdlets oder einer Funktion unterdrückt werden MUSS dies durch eine Zuweisung zur automatischen Variabel \$null erfolgen. Durch die Zuweisung wird dem Leser deutlicher das die Rückgabe ignoriert werden soll anstatt einer Pipeline-Anweisung weit rechts die u.U. leichter übersehen wird.

Beispiel:

	<code>\$null = New-Item -Path 'Test' -ItemType Directory</code>
	<code>New-Item -Path 'Test' -ItemType Directory Out-Null</code>

Code-8

Die automatische Variable \$null MUSS links von einem Vergleichsoperator stehen wenn eine Variable auf NULL geprüft werden soll.

Beispiel:

	<code>\$null -eq \$Array</code>
	<code>\$Array -eq \$null</code>

Code-9

Ein Controller oder eine Funktion MUSS übergebene Argumente vor Ausführung des Codes prüfen. Dazu MUSS eines der Validate-Attribute dem Parameter beigefügt werden oder es MUSS der dedizierte Block PARAMETER CHECK genutzt werden.

Code-10

String Literale die keine Variablen-Expansion enthalten MÜSSEN in Hochkommas gesetzt werden.

Beispiel:

	\$Name = 'Peter Smith'
	\$Name = "Peter Smith"

Code-11

Funktionen oder Cmdlets dürfen NICHT mit ihren Alias aufgerufen werden.

Code-12

Beim Aufruf einer Funktion oder Cmdlet MÜSSEN die Namen der Parameter vollständig ausgeschrieben werden.

Beispiel:

	TestConfigFile -Path c:\script\config.json
	TestConfigFile -p c:\script\config.json

Code-13

Aliase für Parameter von Funktionen oder Cmdlets DÜRFEN NICHT verwendet werden.

Code-14

Beim Aufruf einer Funktion oder Cmdlets MÜSSEN die Argumente mittels der Parameter angegeben werden, die Auswertung über die Position DARF NICHT genutzt werden. Als Ausnahme gilt nur Regel **Code-15**.

Beispiel:

	TestConfigFile -Path c:\script\config.json
	TestConfigFile c:\script\config.json

Code-15

Die folgenden PowerShell Cmdlets DÜRFEN ohne Parameternamen genutzt werden:

Read-Host
Write-Host
Write-Output

Code-16

Nächste Seite.

Code-16

Bei der Nutzung von Funktionen bzw. Cmdlets MÜSSEN Verb, Noun und Parameter in PascalCase formatiert werden.

Beispiel:

	Get-ChildItem -Path D:\Logs
	get-childitem -path D:\Logs

Code-17

Ist der Aufruf einer Funktion sehr lang MUSS das Parameter-Splatting genutzt werden.

Beispiel:

	<pre>\$Parameters = @{ 'ErrorAction' = 'Stop' 'Name' = \$GPO.DisplayName 'PermissionLevel' = 'None' 'TargetType' = 'User' 'TargetName' = \$Trustee 'WhatIf' = \$false } Set-GPPermission @Parameters -Replace</pre>
	<pre>Set-GPPermission -Name \$GPO.DisplayName -Replace -TargetType User - TargetName \$Trustee -PermissionLevel None -WhatIf:\$false -ErrorAction Stop</pre>

Code-17.1

Der Funktionsaufruf SOLLTE direkt in der Zeile nach der Definition der Hashtabelle erfolgen und diese MUSS einer Variablen mit dem Namen \$Parameters zugewiesen werden.

Code-17.2

Kann der Funktionsaufruf nicht direkt nach der Parameters-Hashtabelle erfolgen, weil z.B. die Hashtabelle in mehreren Schritten aufgebaut wird MUSS der Bezeichner der Hashtabelle den Funktionsnamen beinhalten:

\$<FunktionsName>Parameters

Beispiel:

	<pre>\$SetGPPermissionParameters = @{ ... } . . . Set-GPPermission @SetGPPermissionParameters -Replace</pre>
---	--

Code-18

Controller oder öffentliche Funktionen, die Änderungen an produktionskritischen IT-Assets vornehmen MÜSSEN die WhatIf-Methodik unterstützen, also auch einen WhatIf-Parameter zur Verfügung stellen.

Code-19

Pipeline-Konstrukte SOLLTEN vermieden werden aber wenn dann so kurz wie möglich sein.

Code-20

Pipeline-Konstrukte DÜRFEN sich NICHT über mehr als eine Zeile erstrecken.

Code-21

Pipeline-Konstrukte, die eine Schleife darstellen MÜSSEN durch eine for- oder foreach Kontrollstruktur ersetzt werden.

Beispiel:

	<code>\$Users = Import-Csv -Path Users.csv foreach (\$User in \$Users) { Do-Anything \$User }</code>
	<code>Import-Csv Users.csv %{ Do-Anything \$_ }</code>

Die foreach-Schleife ist besser erkennbar und vermeidet die anonyme Variable \$_.

Code-22

Wenn eine foreach-Schleife durch eine Kollektion läuft MUSS die Laufvariable von dem Namen der Kollektion abgeleitet werden d.h. es MUSS der Singular des Namens der Kollektion sein.

Beispiel:

	<code>foreach (\$User in \$Users) {...}</code>
	<code>foreach (\$Account in \$Users) {...}</code>
	<code>foreach (\$u in \$Users) {...}</code>

Es DARF von dieser Regel abgewichen werden wenn der Name der Kollektion gleichzeitig ein Parameter des Controllers/Funktion ist, da in diesem Fall der Bezeichner der Kollektion schon Singular ist. Es wird empfohlen dann die Laufvariable mit \$Item zu bezeichnen.

Code-23

Literele und "Magic Values" MÜSSEN vermieden werden.

Beispiel:

	<code>New-Variable -Name LOGFILE_PATH -Value 'ScriptLog.txt' -Option Constant ... Out-File -FilePath \$LOGFILE_PATH</code>
	<code>Out-File -FilePath 'ScriptLog.txt'</code>

Die Literale sind dann als Konstanten anzusehen und MÜSSEN im Block DECLARATIONS AND DEFINITIONS initialisiert werden.

Code-24

Sind alle Werte einer Hashtabelle aus dem gleichen Wertebereich z.B. E-Mail-Adressen oder AD User-Objekte, wird EMPFOHLEN den Namen der Hashtabelle folgendermaßen zu bilden:

`$<ValueType>Of[<KeyName>]`

Beispiel:

	<code>\$MailAddressOf[\$User]</code>
	<code>\$UserObjectOf[\$samAccountName]</code>

Code-25

Immer wenn auf eine Ressource zugegriffen wird, die außerhalb der eigenen Kontrolle liegt SOLL ein `try {} catch {}` Konstrukt eingesetzt werden.

Formatierung

Form-1

Ein Skript muss die zum Zeitpunkt der Erstellung aktuellen EPS-Vorlagen implementieren.

Form-2

PowerShell Schlüsselwörter MÜSSEN klein geschrieben werden:

begin	break	catch	class
continue	continue	define	do
dynamicparam	else	elseif	end
exit	filter	finally	for
foreach	from	function	if
in	param	process	return
switch	throw	trap	try
until	using	var	while

Die hier aufgelisteten Schlüsselwörter sind nicht unbedingt aktuell und vollständig, eine aktuelle Übersicht findet man unter [PSKEYWORDS].

Form-3

PowerShell Operatoren wie -eq, -ne, -gt etc. pp. MÜSSEN klein geschrieben werden.

Form-4

Die automatischen Variablen der PowerShell MÜSSEN identisch zur Definition von Microsoft geschrieben werden. Eine Liste der verfügbaren Variablen findet man unter [MSAUTOVAR].

Beispiel:

	\$null
	\$NULL

Form-5

Beim Aufruf eines externen Programms MUSS das Dateinamen-Suffix angegeben werden, um es deutlich als externes Binary von einer Funktion oder Cmdlet zu unterscheiden.

Beispiel:

	robocopy.exe
	robocopy

Form-6

Codezeilen DÜRFEN NICHT mit einem Backtick (`) umgebrochen werden. Es MUSS ein anderer Weg gefunden werden um die Zeilen zu verkürzen z.B. Parameter-Splatting.

Form-7

Type Accelerators d.h. Aliase für .NET Datentypen MÜSSEN in Kleinbuchstaben geschrieben werden.

Beispiel:

	<code>[string]\$UserName = 'Peter Smith'</code>
	<code>[Parameter(Mandatory=\$true)] [AllowEmptyString()] [string]\$Path</code>
	<code>[String]\$UserName = 'Peter Smith'</code>
	<code>[STRING]\$UserName = 'Peter Smith'</code>

Von dieser Regel ausgenommen sind die Attribut-Typen wie z.B. Alias, AllowNull, AllowEmptyString etc. pp. Eine Liste der verfügbaren Aliase findet man unter [TYPEACCEL].

Form-8

Quellcode MUSS Textausrichtungen so häufig wie möglich einsetzen. Blöcke mit Textausrichtung SOLLEN ebenfalls alphabetisch sortiert sein.

Beispiel 1:

```
$Domain      = $null      # Several Domain data.
$FailedGroups = @()       # Collect all groups with errors.
$Groups      = $null      # All groups to be processed.
$HostInstance = $null     # Computer- and InstanceID.
```

Hier wird sowohl am Gleichheitszeichen als auch am Kommentarzeichen ausgerichtet. Die Zeilen sind alphabetisch nach dem Variablennamen sortiert.

Beispiel 2:

```
$Parameters = @{
    ErrorAction    = 'Stop'
    Name           = $GPO.DisplayName
    TargetName     = $Trustee
    TargetType     = 'User'
    PermissionLevel = 'None'
    WhatIF         = $false
}
```

Hashtabellen werden Links und am Gleichheitszeichen ausgerichtet. Die Zeilen sind alphabetisch nach den Schlüsselnamen sortiert. Die Textausrichtung und Sortierung erzeugt Codeblöcke die übersichtlich sind, leichter erfasst und wiedererkannt werden können.

Form-9

Nächste Seite.

Form-9

Vor und nach dem Zuweisungsoperator MUSS ein Leerzeichen sein.

Beispiel:

	<code>\$Name = 'Bill Gates'</code>
	<code>\$Name='Bill Gates'</code>

Form-10

Eine Zeile DARF NICHT mit einem Semikolon als Zeilenendezeichen versehen werden.

Form-11

Codeblöcke {...} die in Verbindung mit Schlüsselwörtern wie for, foreach, if, else, switch, try, catch etc. definiert werden MÜSSEN die Start- und Endklammern des Blocks mit identischer Einrückung formatieren.

Beispiel:

	<code>for (...) { ... }</code>
	<code>for (...) { ... }</code>

Form-12

Codeblöcke {...} die einen Funktionskörper definieren MÜSSEN die Start- und Endklammern des Blocks mit identischer Einrückung formatieren.

Beispiel:

	<code>function TestConfigFile { ... }</code>
	<code>function TestConfigFile (<ParameterList>) { ... }</code>

Form-13

Nächste Seite.

Form-13

Besteht der Codeblock {...} aus nur einer Anweisung MUSS er einzellig geschrieben werden.

Beispiel:

	<code>foreach (\$Item in \$Items) { Write-Output \$Item }</code>
	<code>if (\$Date -lt \$Today) { Write-Output \$Result; return }</code>
	<code>foreach (\$Item in \$Items) { Write-Output \$Item }</code>

Konstrukte aus der Regel **Code-5** werden hier als eine Anweisung angesehen. Von dieser Regel ausgenommen sind nur sehr lange Codezeilen.

Form-14

Eine Konstruktion mit zwei einzeiligen Codeblöcken MUSS mit gleicher Ausrichtung sowohl der Schlüsselwörter als auch der Codeblöcke formatiert werden.

Beispiel:

	<code>if (\$IsUserExisting){ ... } else { ... }</code>
	<code>if (\$IsUserExisting) { ... } else { ... }</code>

Kommentare

Comm-1

Der Kommentar MUSS in Englisch sein.

Comm-2

Code MUSS ausreichend kommentiert werden.

Comm-3

Ein Kommentar SOLLTE ein kompletter Satz sein und mit einem Punkt enden, außer er ist sehr kurz dann kann der Punkt weggelassen werden.

Comm-4

Der Kommentar MUSS sofort aktualisiert werden wenn der zugeordnete Code sich ändert.

Comm-5

Codeblöcke die im Editor nicht komplett auf einer Seite dargestellt werden können, sehr lang oder verschachtelt sind MÜSSEN einen End-Kommentar haben.

Beispiel:

	<pre>foreach (\$User in \$Users) { ...(Sehr langer Codeabschnitt) }# End of foreach all Users.</pre>
--	--

Der End-Kommentar MUSS mit # End of beginnen und dem Schlüsselwort der Anweisung gefolgt von einem beschreibenden Text. Dies beinhaltet die Schlüsselwörter mit einem Codeblock wie z.B. function, for, foreach, if und switch.

Durch diese Regel werden hängende schließende Klammern erklärt und dem Leser die Zuordnung erleichtert und die Ebene der Verschachtelung deutlicher gemacht.

Comm-6

Es DARF NICHT kommentiert werden was der Leser sowieso schon sieht sondern das 'Warum', 'Wie' und 'Wieso hier'. Ausnahme davon ist Regel **Comm-5**.

Konstanten

Const-1

Der Bezeichner MUSS so beschreibend wie möglich sein und dabei aber nur so lang wie nötig.

Const-2

Der Bezeichner MUSS aus Substantiven getrennt durch den Unterstrich bestehen.

Const-3

Bezeichner MÜSSEN in Großbuchstaben geschrieben werden.

Const-4

Konstanten MÜSSEN mit dem Cmdlet New-Variable erstellt werden.

Beispiel:

	<code>New-Variable -Name REGEX_VALID_NAME -Value '[a-z0-9_]+\$'-Option Constant</code>
	<code>\$RegexValidName = '[a-z0-9_]+\$'</code>

Const-5

Eine Konstante in einer öffentlichen Bibliothek MUSS eindeutig in allen veröffentlichten Bibliotheken sein. Um dieses Ziel zu erreichen SOLL der Name der Konstanten ein Präfix haben das vom Namen der Bibliothek abgeleitet wird.

Beispiel: Ist der Name der öffentlichen Bibliothek

`LogFunction.ACME.lib.ps1`

so hat die Konstante `AD_TIER0_OU` das Präfix `LogFunction`:

`$LOGFUNCTION_AD_TIER0_OU`

Variablen

Var-1

Ein Bezeichner DARF NICHT aus einem einzelnen Buchstaben bestehen, außer in dem Falle dass er als Zählvariable in einer for-Schleife eingesetzt wird.

Beispiel:

	<code>for (\$i=0; \$i -lt 100; \$i++) { ... }</code>
	<code>\$C = (Get-ChildItem -Filter *.txt).Count</code>

Var-1.1

In Zählschleifen MÜSSEN die Bezeichner, die nur aus einem Buchstaben bestehen, die Buchstaben i,j,k,l,... in Kleinbuchstaben verwenden und sie MÜSSEN beim Verschachteln von Zählschleifen genau in dieser Reihenfolge genutzt werden.

Beispiel:

	<code>for (\$a=0; \$a -lt 100; \$a++) { for (\$b=0; \$b -lt 1000; \$b++) { ... } }</code>
	<code>for (\$k=0; \$k -lt 100; \$k++) { for (\$i=0; \$i -lt 1000; \$i++) { ... } }</code>

Var-2

Ein Bezeichner MUSS so beschreibend wie möglich sein dabei aber nur so lang wie nötig.

Beispiel:

	<code>\$Ver = '3.0.1'</code>
	<code>\$VersionOfThisStrangeSoftwareIs= '3.0.1'</code>

Var-3

Der Bezeichner MUSS aus Buchstaben bestehen und DARF NICHT den Unterstrich enthalten.

Var-4

Der Bezeichner MUSS ein Substantiv sein oder eine Kombination aus diesen. Ausnahme davon ist Regel **Var-1**.

Var-5

Der Bezeichner MUSS in PascalCase formatiert sein. Erlaubte Ausnahmen von dieser Regel sind die Regeln **Var-1** und **Var-6** oder Situationen in denen man konform zu einer Schreibweise aus einer anderen prominenten Quellen sein möchte.

Beispiel:

```
$samAccountName = 'TestUser'
```

Durch die unveränderte Übernahme des AD-Attributs als Bezeichner wird der Code verständlicher weil unmittelbar erkennbar ist das hier Daten aus einer spezifischen Quelle z.B. dem Active Directory von Microsoft, verarbeitet werden.

Var-6

Ein Bezeichner DARF mit einem Akronym beginnen wenn es im Kontext üblich oder so allgemein bekannt ist das es ohne weitere Erklärung genutzt werden kann. Wie z.B. die automatische PowerShell Variable \$PSBoundParameters.

Var-7

Enthält eine Variable einen skalaren Datentyp MUSS der Bezeichner in Singular sein.

Var-8

Enthält eine Variable eine Kollektion MUSS der Bezeichner mit dem Buchstaben 's' enden. Die Hashtabelle wird hier NICHT als Kollektion sondern als ein einzelnes Objekt gesehen und DARF deshalb NICHT den Plural verwenden. Ausnahme davon ist nur Regel **Code-17**.

Var-9

Der Bezeichner DARF Typinformationen (Hungarian Notation) NICHT enthalten.

Beispiel:

	\$NumberOfMembers = 1000
	\$IntNumberOfMembers = 1000

Var-10

Der Bezeichner DARF NICHT Ziffern oder andere Art von Nummerierung enthalten.

Beispiel:

	\$Name1
	\$Room1BuildingA

Var-11

Variablen vom Typ 'Boolean', die das Ergebnis einer Entscheidung enthalten SOLLTEN mit den Wörtern Is, Can, Has, Had etc. beginnen.

Var-12

Enthält eine Variable einen Wert der eine physikalische oder andere Art der Einheit besitzt MUSS diese Einheit das Suffix des Bezeichners sein.

Beispiel:

	\$RetentionPeriod Day \$MaxFileSize GB
---	---

Funktionen

Grundsätzlich sollte der bewährten Unix Philosophie **DOTADIW (Do One Thing And Do It Well)** gefolgt werden d.h. eine Funktion sollte ausschließlich genau eine Sache machen und diese richtig. Es sollte allerdings auch hinreichend geprüft werden ob die zu entwickelnde Funktion nicht vielleicht aus der Kombination schon vorhandener Funktionen realisiert werden kann ("Erfinde das Rad nicht neu.") z.B. durch Funktionen aus einem vorinstallierten Modul von Microsoft oder anderer Hersteller die u.U. in der anzustrebenden Lösung sowieso schon vorausgesetzt werden.

Func-1

Funktionen SOLLEN elementar und einfach sein.

Func-2

Funktionsbezeichner MÜSSEN mittels des Verb-Noun Muster benannt werden.

Funktionsbezeichner unterscheiden sich allerdings in der Formatierung in Abhängigkeit ob sie in einer privaten Bibliothek, einem Controller oder in einer öffentlichen Bibliothek definiert sind.

Func-2.1

Funktionsnamen in Controllern oder privaten Bibliotheken MÜSSEN ohne Bindestrich zwischen Verb und Noun geschrieben werden.

Beispiel:

	<pre>function TestConfigFile { ... }</pre>
	<pre>function Test-ConfigFile { ... }</pre>

Der Nachteil der Schreibweise ohne Bindestrich, das manche Editoren dies dann u.U. nicht speziell hervorheben, wird durch den Vorteil aufgehoben das ganz explizit dargestellt wird dass diese Funktion Teil des Controllers selber ist.

Func-2.2

Funktionsbezeichner in öffentlichen Bibliotheken MÜSSEN nach dem folgenden Schema aufgebaut werden:

<Verb>-<CompanyPrefix><Noun>[V<MajorVersion>]

Func-2.2.1

Das <CompanyPrefix> MUSS eine 2-5 stellige Abkürzung des Unternehmens sein und MUSS in Großbuchstaben geschrieben sein. Es ist angeraten das Präfix identisch mit einem eventuell schon extern oder intern genutzten zu wählen.

Func-2.2.2

Eine neue Major-Version größer als '1' MUSS mit dem Versions-Suffix "V<MajorVersion>" kenntlich gemacht werden. Dies verhindert Kompatibilitäts- und Produktionsprobleme bei der Nutzung von Funktionen aus öffentlichen Bibliotheken.

Func-2.3

Funktionsbezeichner in öffentlichen Bibliotheken MÜSSEN ein Verb haben dass dem Approved Verb-Standard von Microsoft folgt. Welche Verben gültig sind findet man aktuell unter [MSAPPRVERB].

Func-2.4

Das Noun des Bezeichners MUSS Singular sein.

Func-2.5

Verb und Noun des Bezeichners MÜSSEN ausschließlich Buchstaben enthalten.

Func-2.6

Verb und Noun des Bezeichners MÜSSEN in PascalCase formatiert sein.

Func-3

Der Funktionsbezeichner DARF NICHT den Typ der zurückgegeben Daten enthalten (Hungarian Notation)

Func-4

Die Parameter einer Funktion MÜSSEN im Funktionskörper mittels des Schlüsselwortes param() definiert werden und DARF NICHT im Funktionskopf erfolgen.

Beispiel:

	<pre>function TestUser { [CmdletBinding()] param () ... }</pre>
	<pre>function TestUser (\$User) { ... }</pre>

Func-5

Funktionen MÜSSEN den folgenden Aufbau haben.

```
function <Name>
{
    + <FUNCTION HELP>
    + <PARAMETERS>
    + <PARAMETER CHECK>
    + <DECLARATIONS AND DEFINITIONS>
    + <FUNCTION CODE>

}# End of function <Name>
```

FUNCTION HELP

Dieser Block ist ein mehrzeiliger PowerShell Kommentar `<# ... #>`, der die Entwickler-Hilfe zur Funktion beinhaltet. Der Block besteht aus verschiedenen Sektionen, die durch ein dotted Keyword eingeleitet werden.

Func-5.1

Der Block stellt einen wesentlichen Beitrag zur Dokumentation der Funktion dar und MUSS angegeben werden.

Func-5.1.1

Es gibt einzeilige und mehrzeilige Sektionen, bei einzeiligen Sektionen MUSS der Wert in der gleichen Zeile des Keywords stehen, bei mehrzeiligen MUSS der erste Wert auf der nächsten Zeile eingerückt stehen und jeder Wert auf einer eigenen Zeile beginnen, Werte können aber mehrzeilig sein.

```
.<KEYWORD> [Value]    Einzeilige Sektion
.<KEYWORD>             Mehrzeilige Sektion
    {Values}
```

Func-5.1.2

Ein Keyword MUSS in Großbuchstaben geschrieben werden.

Func-5.1.3

Eine Sektion die OPTIONAL ist und keinen Wert hat DARF komplett weggelassen werden.

Func-5.1.4

Die folgenden Keywords, optionale oder verpflichtende, SOLLEN enthalten sein und MÜSSEN die für sie hier spezifisch definierten Regeln erfüllen.

```
[.SYNOPSIS
    <SYNOPSIS>]
```

Eine sehr kurze Inhaltsangabe.

```
.DESCRIPTION
    <DESCRIPTION>
```

Eine hinreichend umfangreiche Beschreibung der Aufgabe und Nutzung der Funktion. Dies stellt die Dokumentation der Funktion dar und MUSS vorhanden sein.

```
.INPUTS
    None | {INPUT}
    [You cannot pipe input to this function]
```

Die Angabe der Input-Datentypen und eine kurze Beschreibung der Semantik der Parameter.

Hat die Funktion keine Parameter MUSS hier das Wort None stehen.

Unterstützt die Funktion keine Pipeline-Verarbeitung MUSS der definierte Hinweis angegeben werden.

```
.OUTPUT
    NONE | {OUTPUT}
```

Beschreibung des Rückgabetyps. Hat die Funktion keine Ausgabe MUSS hier None stehen.

Jeder einzelne Input- oder Output-Eintrag MUSS mindestens über zwei Angaben verfügen, den Datentyp und einen Beschreibungstext.

Beispiel:

`.INPUTS`

System.String

The log messages, a log type and an action type.

`[You cannot pipe input to this function.]`

[.NOTES

`<NOTES>]`

Enthält zusätzliche Informationen, die für das Verständnis des Codes und der Nutzung der Funktion hilfreich sein können.

Eine Funktion einer öffentlichen Bibliothek SOLL hier dokumentieren, ob sie den ErrorAction-Parameter unterstützt. Dies kann z.B. durch eine der folgenden Formulierungen erfolgen:

ErrorAction not supported, the function is creating terminating errors.

ErrorAction supported, the function is creating non-terminating errors.

{.PARAMETER <NAME>

`<DESCRIPTION>}`

Die Parameter der Funktion und eine kurze Beschreibung. Hat der Parameter einen Standardwert SOLL dieser hier durch eine Formulierung wie The default value is <Value> beschrieben werden.

Eine öffentliche Bibliothek KANN weiterhin noch die folgenden Sektionen enthalten:

{.EXAMPLE

`<EXAMPLE>}`

Beispiel für den Aufruf der Funktion mit Parametern und der Ausgabe, die u.U. zurückgegeben wird. Jedes Beispiel MUSS eine eigene Sektion sein.

{.LINK

`<LINK>}`

Verweise zu Ressourcen, die zu der Funktion in Beziehung stehen.

Werden eine oder beide dieser Sektionen genutzt MÜSSEN sie zwischen den Sektionen NOTES und PARAMETER eingefügt sein.

PARAMETERS

Func-5.2

Der Block enthält die Definitionen der Parameter der Funktion. Hat die Funktion keine Parameter MUSS ein leerer Block eingefügt werden:

`# PARAMETERS`

`# NA`

andernfalls

`# PARAMETERS`

`<ParameterDefinition>`

Wie die <ParameterDefinition> im Detail auszusehen hat entnehme man dem Abschnitt **Parameter**.

PARAMETER CHECK

Das Fail Fast-Prinzip wird durch die frühzeitige Überprüfung der Argumente, die der Funktion übergeben werden, befolgt. Je besser die Definition des erlaubten Wertebereichs und dessen Überprüfung, um so einfacher wird die Nutzung der Parameter. Die prominente Platzierung der Überprüfung an den Anfang des Funktionskörpers erhöht die Klarheit und Lesbarkeit.

Func-5.3

Die zur Laufzeit übergebenen Parameter MÜSSEN auf Richtigkeit geprüft werden bevor der eigentliche Code abgearbeitet wird.

Func-5.3.1

Es SOLLEN vorrangig die von der PowerShell zur Verfügung gestellten Validate-Attribute genutzt werden. Sind die Attribute in der Parameter-Deklaration hierfür nicht ausreichend MUSS die Überprüfung mittels des PARAMETER CHECK Blocks erfolgen.

Func-5.3.2

Hat die Funktion keine Parameter oder braucht keine weitere Überprüfung MUSS ein leerer Block eingefügt werden:

```

                                # PARAMETER CHECK
                                # NA
andernfalls
                                # PARAMETER CHECK
                                {ParameterCheck}

```

DECLARATIONS AND DEFINITIONS

Die Definition und Deklaration von Variablen und Argumenten an einer dedizierten und prominenten Position in der Funktion verdeutlicht die Nutzung und Wichtigkeit der Elemente.

Func-5.4

Werden keine Deklarationen/Definitionen genutzt MUSS ein leerer Block der folgenden Form eingefügt werden:

```

# DECLARATIONS AND DEFINITIONS
# ARGUMENTS
# NA

# VARIABLES
# NA

```

andernfalls MÜSSEN Variablen und Argumente in getrennten Sektionen deklariert bzw. definiert werden.

```

# DECLARATIONS AND DEFINITIONS
# ARGUMENTS
<ArgumentDefinitions>

# VARIABLES

```


<VariableDefinitions>

Die Sektionen ARGUMENTS und VARIABLES MÜSSEN eingerückt werden.
Zur Definition von Konstanten und Argumenten informieren die entsprechende Abschnitte **Konstanten** bzw. **Parameter**.

FUNCTION CODE

Func-5.5

Der Code der Funktion MUSS mit folgenden Start-Kommentar eingeleitet werden:

```
# FUNCTION MAIN CODE
```

Func-6

In Funktionen DÜRFEN Konstanten NICHT deklariert/definiert werden.

Func-7

Das Schlüsselwort return SOLL nur zum vorzeitigen Beenden einer Funktion genutzt werden und DARF NICHT am Ende einer Funktion eingesetzt werden.

Beispiel:

	<pre>function TestUser { ... if (\$HasError) { return } ... }</pre>
	<pre>function TestUser { ... return # Letzte aktive Codezeile. }</pre>

Func-8

Das Schlüsselwort return DARF NICHT mit der Rückgabe von Objekten verbunden werden.

Beispiel:

	<pre>function TestUser { ... return \$Result ... }</pre>
---	--

Func-9

Eine Funktion MUSS Rückgabewerte mit dem Cmdlet Write-Output zurückgeben.

Func-10

Die Rückgabe einer Funktion MUSS im Falle eines Skalars in einer Variable mit dem Namen \$Result, im Falle eine Collection in \$Results gespeichert werden. Eine Hashtabelle ist keine Collection sondern ein Objekt.

Func-11

Funktionen DÜRFEN NICHT Formatierungen an den Objekten, die sie zurückgeben, ausführen d.h. es DÜRFEN NICHT Format-Cmdlets zur Ausgabe verwendet werden.

Func-12

Funktionen DÜRFEN NICHT auf Variablen des aufrufenden Scopes schreibend zugreifen.

Beispiel:

	<code>\$Script:NumberOfUsers += 1</code>
---	--

Parameter

Unter dem Begriff Parameter sollen hier spezifisch nur die formalen Argumente von Controllern und Funktionen verstanden werden.

Para-1

Der Bezeichner MUSS in Singular geschrieben werden.

Para-2

Der Bezeichner MUSS in Pascal Notation formatiert sein.

Para-3

Der Bezeichner MUSS ausschließlich aus Buchstaben bestehen

Para-4

Der Bezeichner DARF Typinformation (Hungarian Notation) NICHT enthalten.

Para-5

Kann ein Parameter mit den Validate-Attributen überprüft werden, so MUSS dies implementiert werden.

Para-6

Der Name eines Parameters SOLLTE mit existierenden Bezeichnungen von Microsoft konform sein.

Beispiel:

	TestNetwork -Computer 'File1' FindMember -Identity 'Meyer'
	TestNetwork -Host 'File1' FindMember -Account 'Meyer'

Dies bezieht sich natürlich auch auf andere Hersteller z.B wenn man Skripts im Storage Umfeld entwickelt und Module von vSphere oder NetApp genutzt werden.

Am besten orientiert man sich also an schon vorhandenen Cmdlets oder Funktionen, die in einer bestimmten Domäne allgemein genutzt werden ("Don't be too smart"-Prinzip) und erfindet keine neuen Konventionen.

Para-7

Ein Parameter eines Controllers oder Funktion DARF NICHT auf der rechten Seite einer Zuweisung stehen. d.h. der Parameter muss während der gesamten Laufzeit des Controllers oder der Funktion das an ihn übergebene Argument beibehalten.

Beispiel:

	<pre>function ConvertName { param ([string]\$Name) ... \$Name = "\$Name_Admin" ... }</pre>
---	---

Para-7.1

Soll der Name eines Parameters, aus Gründen der Lesbarkeit oder weil ein anderer Bezeichner ungünstig oder zu unpassend wäre, trotzdem genutzt werden und man Zuweisungen an diesen Parameter machen möchte so MUSS eine Argument-Variable deklariert werden. Die Deklaration MUSS im Block DECLARATIONS AND DEFINITIONS in der Sektion ARGUMENTS erfolgen und es MUSS eines der folgenden zwei Namens-schemata für die Variable genutzt werden:

```
# DECLARATIONS AND DEFINITIONS
# ARGUMENTS
$<ParameterName>Argument = $null
```

oder

```
# DECLARATIONS AND DEFINITIONS
# ARGUMENTS
$<ParameterName>Arg = $null
```

Durch das Anhängen des Suffix `Argument` oder `Arg` an den Parameternamen wird sofort deutlich dass das übergebene Argument modifiziert werden soll, der ursprüngliche Parameter dabei aber nicht verändert wird.

Para-8

Hat der Controller bzw. Funktion keine Parameter MUSS ein `<ParametersDefintion>` Block der folgenden Form eingefügt werden:

```
# PARAMETERS
[CmdletBinding()]
param ()
```

andernfalls

```
# PARAMETERS
[CmdletBinding()]
param
(
    {Parameter}
)
```

Para-8.1

Ein Parameter der `{Parameter}` Liste MUSS nach dem folgenden definierten Schema erstellt werden und alle damit verbundenen Regeln des Aufbaus beinhalten. Eine Parameter-Definition besteht somit aus den folgenden Blöcken:

```
+ [PARAMETER ATTRIBUTES]
+ [VALIDATE ATTRIBUTES]
+ [OTHER ATTRIBUTES]
+ <TYPE AND NAME>
```

Para-8.1.1

Alle Parameter-Attribute MÜSSEN am Beginn der Definition vorhanden sein.

Para-8.1.2

Validate-Attribute sind optional, sofern sie eingesetzt werden MÜSSEN sie aber direkt auf die Parameter-Attribute folgen.

Para-8.1.3

Alle sonstigen Attribute MÜSSEN nach den Validate-Attributen folgen.

Para-8.1.4

Das Attribut `[Alias()]` DARF NICHT verwendet werden.

Para-8.1.5

Nächste Seite.

Para-8.1.5

Der Typ des Parameter MUSS explizit definiert werden:

```
[<ParameterTyp>]$<ParameterName>
```

Para-9

Unterstützt der Controller oder die Funktion den -WhatIf Risk-Mitigation Schalter MUSS dies im CmdletBinding-Attribut aktiviert sein:

```
[CmdletBinding(SupportsShouldProcess=$true)]
```

Versionierung

Der EPS-Standard folgt weitgehend dem Semantic Versioning Standard. Die vollständige Spezifikation dazu findet sich in [SEMVERS].

Ver-1

Die folgenden Code Container MÜSSEN eine Versions-Nummer beinhalten:

- + Controller
- + Funktionen in einer öffentlichen Bibliothek

Ver-2

Der Versionsbezeichner MUSS aus drei positiven Integer Werten in der folgenden Form bestehen:

`<Major>.<Minor>.<Patch>`

Ver-3

Die Inkrementierung DARF Nummern NICHT überspringen.

Ver-4

Eine Major-Version '0' ist stets als Testversion anzusehen und SOLL NICHT in der Produktion eingesetzt werden. Wenn doch obliegt die Verantwortung beim Nutzer, das keinerlei Schäden in der Produktion entstehen.

Ver-5

Ist eine Major-Version größer als 0 und aktuell in einer Testphase MUSS dies durch das Suffix -alpha oder -beta am Patch-Level kenntlich gemacht werden z.B. 1.0.4-alpha.

Ver-6

Ein Skript das in zwei verschiedenen Major Versionen vorliegt ist prinzipiell als nicht austauschbar anzusehen d.h. es MUSS vom Nutzer ausführlich getestet werden bevor es in der Produktion eingesetzt wird. Der Autor des Skripts macht durch den Wechsel der Major Version unmissverständlich klar das Inkompatibilitäten zu erwarten sind.

Ver-7

Die Major-Version MUSS in den folgenden Fällen inkrementiert werden:

- + Ein Parameter wird entfernt.
- + Ein Parameter wird umbenannt.
- + Der Typ eines Parameters ändert sich.
- + Der Objekttyp der Ausgabe ändert sich
- + Ein neuer Parameter hat eine Wechselwirkung mit einem bereits vorhandenen Parameter
- + Der selbe Parameter veranlasst ein geändertes Verhalten.
- + Der Aufbau oder das Format des Log ändert sich.
- + Vorhandene Logeinträge werden durch neue ersetzt.
- + Der Standard-Wert eines Parameters ändert sich.
- + Der Parameter-Check eines Parameters ändert sich.

Ver-8

Neue Funktionalität, Verbesserung der Performance oder das Refactoring MÜSSEN durch eine neue Minor-Version angezeigt werden, sofern die Änderung nicht die Fälle aus Regel **Ver-7** betrifft.

Ver-9

Fehlerbeseitigungen MÜSSEN durch eine neue Patch-Version angezeigt werden.

Konfiguration

Conf-1

Variablen und Konstanten innerhalb eines Controller DÜRFEN NICHT für die Konfiguration einer spezifischen Einsatzsituation geändert werden.

Ist dies der Fall, so ist das ein klares Anzeichen dafür dass der Controller eine externe Konfigurationsdatei benötigt.

Conf-2

Benötigt ein Controller eine externe Konfigurationsdatei MUSS das JSON-Format genutzt werden.

Conf-3

Die Parameter der Konfigurationsdatei MÜSSEN dokumentiert werden, entweder im Controller in der NOTES-Sektion des Blocks COMMENT BASED HELP oder in einer separaten Hilfe-Datei.

Conf-4

Die Konfigurationsdaten MÜSSEN im Controller über die Variablen `$Config` oder `$Configuration` verfügbar gemacht werden.